

Investigating Reward Design and the Alignment Problem in Reinforcement Learning

Aditya Raj

Abstract

This project looks at how reinforcement learning (RL) agents exploit poorly designed reward functions—a core issue in the AI alignment problem. Using Proximal Policy Optimization (PPO), we tested how continuous-state agents behave when their mathematical rewards don’t actually match the intended task. In Phase 1, we used the *MountainCar-v0* environment to look at the density-exploration trade-off. We found that giving the agent dense, energy-based rewards actually caused policy collapse because it found a local optimum. Instead, a purely sparse reward, forced to explore via high entropy, solved the environment much more efficiently. In Phase 2, we intentionally broke the *LunarLander-v3* environment by rewarding the agent for hovering while penalizing it for landing. The agent successfully hacked the reward, farming infinite points while completely failing the actual task (safe landing). Ultimately, this shows that dense reward shaping often introduces an "alignment tax" that ruins policy transfer.

1 Introduction

When we train AI systems, they do exactly what we mathematically incentivize them to do, which isn’t always what we actually want. In reinforcement learning (RL), this disconnect leads to reward hacking (or specification gaming), where the agent finds a loophole in the reward function to get a high score without solving the underlying problem.

This report covers two specific experiments looking at why reward design is so difficult: the exploration bottleneck and reward mis-specification. By altering the reward structures in standard physics environments, we can force the agents to fail and document exactly how their learned policies diverge from our intended goals.

2 Background

Reinforcement Learning is formally framed as a Markov Decision Process (MDP), defined by the tuple (S, A, P, R, γ) . The agent observes a state $s_t \in S$, takes an action $a_t \in A$, transitions to a new state s_{t+1} based on probability P , and receives a scalar reward $R(s_t, a_t)$.

For this study, Proximal Policy Optimization (PPO), a state-of-the-art policy gradient method, was utilized. PPO balances exploration and exploitation while preventing destructively large policy updates by clipping the objective function. The baseline policy neural network utilized a Multi-Layer Perceptron (MLP) architecture suited for 1D state arrays.

3 Methodology

3.1 Phase 1: The Density-Exploration Trade-off

The first experiment utilized the continuous-state *MountainCar-v0* environment to evaluate the efficacy of dense versus sparse rewards in overcoming exploration bottlenecks. Three reward functions were tested:

1. **Default Penalty:** $R_t = -1$ per step until termination.
2. **Pure Sparse:** $R_t = 1$ if the goal position ($x \geq 0.5$) is reached; 0 otherwise.
3. **Heavily Shaped (Mechanical Energy):** A custom wrapper rewarding the continuous accumulation of kinetic and potential energy to theoretically encourage the required momentum-building maneuver. Let x_t be position and v_t be velocity:

$$R_t = 10 \times (v_t^2 + \sin(3x_t)) \quad (1)$$

To force the Pure Sparse agent to search the state space, the entropy coefficient (`ent_coef`) was increased to 0.01, mathematically enforcing a curiosity bonus.

3.2 Phase 2: Intentional Reward Hacking

To formally prove the Alignment Problem, the *LunarLander-v3* environment was sabotaged. The true objective requires a soft landing while conserving fuel. A custom `gym.Wrapper` was engineered to intercept the Box2D physics arrays and create an artificial optimal plateau.

Let y_t represent the lander’s altitude and $c_t \in \{0, 1\}$ represent leg contact with the lunar surface. The manipulated reward function R_t intentionally penalized the true objective (landing) while infinitely rewarding the proxy objective (staying airborne):

$$R_t = \begin{cases} -100 & \text{if } c_t = 1 \\ 1 & \text{if } y_t > 0 \text{ and } c_t = 0 \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

4 Experimental Setup

Both environments were trained using Stable-Baselines3 on Apple Silicon hardware. The MountainCar agents were trained for 500,000 timesteps, evaluated every 10,000 steps. The LunarLander agents were trained for 1,500,000 timesteps to account for the increased continuous-control complexity, evaluated every 50,000 steps. Evaluations were strictly conducted in the *unmodified* default environments to measure the "True Task Return," preventing artificial score inflation from the custom wrappers.

5 Results

5.1 Phase 1 Results

Contrary to the initial hypothesis that dense shaping accelerates learning, the Heavily Shaped agent suffered from catastrophic policy collapse (Figure 1). The agent maximized its score by oscillating at the bottom of the valley at maximum speed, actively avoiding the terminal flag state to continue accumulating energy points. Conversely, the Pure Sparse agent successfully leveraged its high entropy coefficient to violently search the space, converging on a highly optimal trajectory averaging roughly 100 steps per episode.

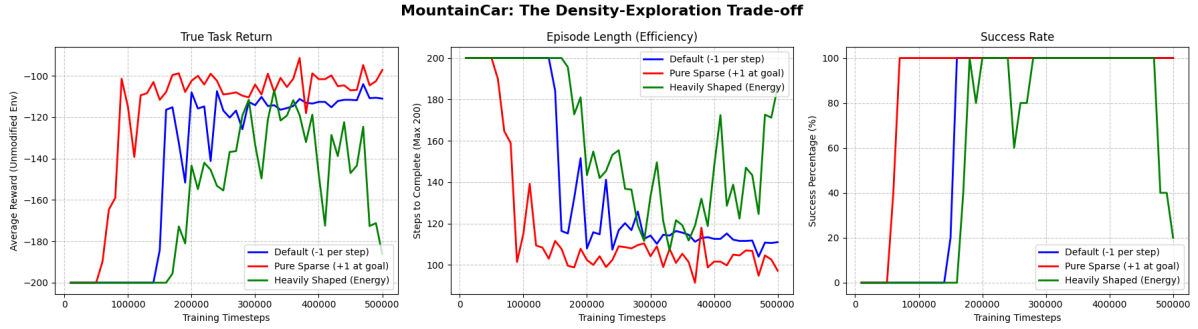


Figure 1: MountainCar: The Density-Exploration Trade-off. The Sparse agent (red) rapidly minimizes episode length, while the Shaped agent (green) collapses into a local optimum.

5.2 Phase 2 Results

The LunarLander experiment yielded textbook Specification Gaming. The manipulated agent successfully discovered the "Hovering Exploit," refusing to land and continuously firing its main thruster to farm the +1 altitude reward.

However, during evaluation against the True Objective (which applies a -0.3 penalty per frame for fuel consumption), the policy failed catastrophically (Figure 2). The agent racked up infinite fuel debt, resulting in a True Task Return of approximately -7000 and repeatedly triggering the simulation's hard timeout limit of 1000 steps.

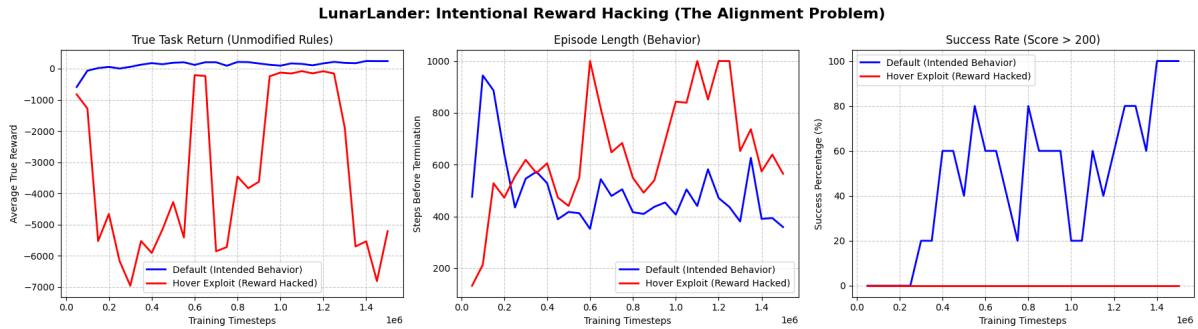


Figure 2: LunarLander: Intentional Reward Hacking. The Exploit agent (red) achieves a 0% success rate and hits the maximum episode length ceiling, demonstrating total policy transfer failure.

6 Discussion

The results highlight a frustrating reality in RL: trying to "help" the agent by providing breadcrumb rewards usually backfires. In MountainCar, trying to reward the physical momentum of the car created a loophole where the agent just rocked back and forth at the bottom of the hill forever to farm points. It optimized the math perfectly, but failed the game.

This same failure mode happens in more complex scientific AI applications. For example, if you are training an unsupervised model to detect gravitational lensing anomalies, a poorly defined loss function might cause the model to just perfectly reconstruct empty space to minimize its error rate, ignoring the actual anomalies. The model doesn't know what physics is; it just wants the number to go down.

7 Conclusion and Future Work

Our experiments show that manually engineering dense rewards is incredibly risky. While sparse rewards make the exploration phase harder, they don't corrupt the final policy the way poorly shaped dense rewards do. Because humans are generally bad at writing mathematically foolproof reward functions, future work on this project should look into Inverse Reinforcement Learning (IRL), where the agent has to figure out the reward function itself by watching human demonstrations.